

PROCESADORES DE LENGUAJES

PRÁCTICA 1: Diseño del Lenguaje



**UNIVERSIDAD
DE GRANADA**

**Antonio Cantillo Molina
Jaime Corzo Galdó
Laura Salas López
Sara Martín Rodríguez**

ÍNDICE

INTRODUCCIÓN: Breve Descripción del Lenguaje Asignado.....	3
DESCRIPCIÓN FORMAL DE LA SINTAXIS DEL LENGUAJE (BNF).....	4
DEFINICIÓN DE LA SEMÁNTICA EN LENGUAJE NATURAL.....	5
IDENTIFICACIÓN DE LOS TOKENS.....	6

INTRODUCCIÓN: Breve Descripción del Lenguaje Asignado

El lenguaje asignado es BABBA. Este incluye:

- Sintaxis inspirada en lenguaje C.
- Palabras reservadas en Castellano.
- La estructura de datos considerada como tipo elemental son los arrays 1D y 2D.
- Como subprogramas, emplearemos los procedimientos.
- Como estructura de control adicional, usaremos case/switch.

DESCRIPCIÓN FORMAL DE LA SINTAXIS DEL LENGUAJE (BNF)

Identificadores

<letra> ::= a | b | c | d | ... | z | A | B | C | D | ... | Z

<numero> ::= 0 | 1 | 2 | 3 | ... | 9

<identificador> ::= <letra> <letra_numero>

<letra_numero> ::= <letra> <letra_numero>
| <numero> <letra_numero>
|

<caracter_ascii> ::= *Todos los caracteres de la tabla ASCII excepto " *

<cadena_ascii> ::= <caracter_ascii> <cadena_ascii>
|

<cadena> ::= " <cadena_ascii> "

<entero> ::= <numero> <entero>
| <numero>

<caracter> ::= ' <caracter_ascii> '

<booleano> ::= true
| false

<vector_1D> ::= { <lista_expresiones> }

<lista_vector_1D> ::= <vector_1D>
| <vector_1D> ", " <lista_vector_1D>

<vector_2D> ::= { <lista_vector_1D> }

<real> ::= <entero> "." <entero>
| "." <entero>
| <entero> "."

Datos y Operaciones

<tipo_dato> ::= "entero" | "real" | "caracter" | "booleano"

<operador_unario> ::= "-" | "!" | "++" | "--" | "&"

<operador_binario> ::= "+" | "-" | "*" | "/" | "%" | "***" | ">" | ">=" | "<" | "<=" | "+=" | "-=" |
"*= " | "/="

<operador_logico> ::= "&&" | "||" | "==" | "!="

Sentencias

Declaración de variables

<tamaño> ::= <entero>

<lista_declaraciones> ::= <declaracion>
| <lista_declaraciones> <declaracion>

<declaracion> ::= <tipo_dato> <lista_id> ","

<lista_id> ::= <identificador>
| <identificador> "," <lista_id>
| <identificador> "[" <tamaño> "]"
| <identificador> "[" <tamaño> "]" "," <lista_id>
| <identificador> "[" <tamaño> "]" "[" <tamaño> "]"
| <identificador> "[" <tamaño> "]" "[" <tamaño> "]" "," <lista_ids>

Lista variables

<lista_vars> ::= <lista_variables> "," <identificador>
| <identificador>
|

<lista_vars_opt> ::= <lista_vars>

Información salida por pantalla

<lista_exp_o_cad> ::= <lista_expresiones_cadenas> "," <exp_o_cad>
| <exp_o_cad>

<exp_o_cad> ::= <expresion>
| <cadena>

<lista_exp_o_cad_escritura> ::= <exp_o_cad>
| <lista_exp_o_cad_escritura> "," <exp_o_cad>

Sentencia Asignación

<asignacion> ::= <identificador> "=" <exp_o_cad>
| <identificador> "=" "{" <lista_exp_o_cad> "}"
| <identificador> "=" "{" <asignacion_2D> "}"

<asignacion_2D> ::= "{" <lista_exp_o_cad> "}"
| "{" <lista_exp_o_cad> "}" "," <asignacion_2D>

Sentencia Entrada

<sentencia_entrada> ::= "lectura" <lista_vars_opt>

Sentencia Salida

<sentencia_salida> ::= "escritura" <lista_exp_o_cad_escritura>

Sentencia If else

<sentencia_if> ::= "si" "(" <expresion> ")" <sentencia>
| "si" "(" <expresion> ")" <sentencia> "si no" <sentencia>

Sentencia While

<sentencia_while> ::= "mientras" "(" <expresion> ")" "hacer" <sentencia>

Sentencia Case/Switch

<cabecera_case> = "(" <expresion> ")" "{"

<sentencia_case> ::= "segun" <cabecera_case> <lista_casos> "}"

<lista_casos> ::= <lista_casos> <caso>
| <caso>

<caso> ::= "caso" <entero> ":" <sentencia> "parar" " ;"
| "caso" <entero> ":" <sentencia>
| "otro" ":" <sentencia>
| "otro" ":" <sentencia> "parar" " ,"

Bloque

<inicio_de_bloque> ::= "{"
<fin_de_bloque> ::= "}"

<bloque> ::=
<inicio_de_bloque><cabecera_bloque><lista_sentencias><fin_de_bloque>

<cabecera_bloque> ::= "locales" <inicio_de_bloque> <lista_decls> <fin_de_bloque>
| <seccion_procs>
| "locales" <inicio_de_bloque> <lista_decls> <fin_de_bloque>
<seccion_procs>
|

Sentencia GENERAL

```
<sentencia> ::= <bloque>
                | <asignacion> “,”
                | <sentencia_if >
                | <sentencia_case>
                | <sentencia_while >
                | <sentencia_entrada> “.”
                | <sentencia_salida> “,”
                | <llamada_proc> “,”
                | <expresion> “,”

<lista_sentencias> ::= <sentencias> <lista_sentencias>
                |
```

Subprogramas: PROCEDIMIENTOS

```
<parametro> ::= <tipo_dato> <identificador>
                | <tipo_dato> “&” <identificador>
                | <tipo_dato> <identificador> “[“ “]”
                | <tipo_dato> <identificador> “[“ “]” “[“ “]”

<lista_parametros> ::= <parametros>
                |

<parametros> ::= <parametro>
                | <parametros> “,” <parametro>

<seccion_procs> ::= <declar_proc>
                | <seccion_procs> <declar_proc>

<declar_proced> ::= <cabecera_proced> <bloque>

<cabecera_proced> ::= “procedimiento” <identificador> ( <lista_parametros> )

<llamada_proc> ::= <identificador> “(“ <lista_arg_call_opt> “)”

<lista_arg_call_opt> ::= <lista_arg_call>
                |

<lista_arg_call> ::= <arg_call>
                | <lista_arg_call> “,” <arg_call>

<arg_call> ::= <exp_o_cad>
```

Programa

<cabecera_programa> ::= "programa"
<programa> ::= <cabecera_programa> <bloque>

Expresiones

<expresion> ::= <unario>
 | <exp_o_cad> <operador_binario> <exp_o_cad>
 | <exp_o_cad> <operador_logico> <exp_o_cad>

<unario> ::= <postfijo>
 | <operador_unario> <unario>
 | <unario> <operador_unario>

<postfijo> ::= <primario>
 | <identificador> <indices>

<indices> ::= <indice>
 | <indice> <indice>

<indice> ::= "[" <expresion> "]"

<primario> ::= "(" <expresion> ")"
 | <entero>
 | <real>
 | "true"
 | "false"
 | <identificador>

DEFINICIÓN DE LA SEMÁNTICA EN LENGUAJE NATURAL

VECTORES

En nuestro lenguaje asignado la estructura de datos considerada como tipo elemental son los **arrays 1D y 2D**. A continuación detallamos sus características.

Primero vamos a detallar que es una <expresion>: Es cualquier valor, ya sea variable o constante, que puede también ser fruto de una operación unaria, binaria o con un operador lógico. Además puede ir o no entre paréntesis para alterar la precedencia de los operadores.

En consecuencia una <lista_expresiones> no es más que la consecución de una o más expresiones separadas por comas. Un ejemplo de una <lista_expresiones> sería:

$x + 3, y * 2, z, 7$

Donde:

($x + 3$) es una expresión,
luego viene , $y * 2$,
luego , z ,
luego , 7 .

Declaración de los arrays 1D y 2D

Para declararlos se procede como sigue:

```
<vector_1D> ::= { <lista_expresiones> }  
<lista_vector_1D> ::= <vector_1D> | <vector_1D>, <lista_vector_1D>  
<vector_2D> ::= { <lista_vector_1D> }
```

Por lo que un array 1D no es más que una <lista_expresiones> entre corchetes, lo que le confiere la naturaleza de vector, y para formar una matriz, es decir, un array 2D hay que formar una <lista_vector_1D>, es decir, un secuencia, separada por comas, de array 1D, cada uno de estos representa una fila de la matriz.

Acceso a un elemento

Para acceder a un elemento concreto de un array 1D o un array 2D usamos el:

```
<operador_vector> ::= [<entero>] <operador_vector>  
                      | [<identificador>] <operador_vector>  
                      |
```

En el que podemos especificar entre corchetes la posición a la que se desea acceder, bien con un entero o con una variable o constante que tenga un valor entero. Observando su funcionamiento podemos ver que permite acceso a vectores de dimensión mayor que 2. En nuestro lenguaje con dimensión 2 es suficiente.

Posteriormente este <operador_vector> será usado junto con un identificador para determinar el vector al que queremos acceder.

Constantes de tipo array

También es posible tener una constante de tipo array, ya que uno de los valores que puede tomar <constante> es el de <vector_1D> o <vector_2D>.

SENTENCIAS

Las sentencias son esenciales para poder expresar y utilizar distintas funcionalidades. Todas las sentencias irán terminadas por un símbolo ; para indicar su final.

Para empezar, saber declarar variables sobre las que poder realizar operaciones. Definiendo el tipo del dato y el nombre que le queremos poner, se crea una variable de dicho tipo.

entero numero;	(Crea una variable llamada entero que almacena valores de tipo entero)
booleano v_o_f;	(Crea una variable de nombre "v_o_f" que almacena valores de tipo booleano)

Para entender como realizar todas las sentencias posibles en el lenguaje de programación, usaremos como ejemplos variables de nombre "x" e "y":

- Si ponemos **x = y;** , el valor de x se sustituye por el de y. Es decir, se entiende como que **el valor de x pasa a ser el de y.**
O también podemos asignar a una variable el valor de una expresión: **x=y+4;**
- Con la palabra reservada "**lectura**" se realiza la función del "**cin**" de lenguajes de programación como C. El uso sigue siendo el mismo: **lectura x,y;**

Este ejemplo, en el programa se interpreta como que los valores introducidos y pedidos por pantalla, el primero se guarda en x y el segundo en y.

- Para la impresión por pantalla de valores, utilizamos la palabra reservada "**escritura**":

escritura x;	(Escribe el valor de x)
escritura y+4;	(Escribe el valor de la expresión y+4)

- Para un **sentencia condicional o if-else**:

```
si (expresión condicional) {  
    // bloque instrucciones  
} si no {  
    // bloque instrucciones  
}
```

- Para una **sentencia while**:

```
mientras (expresión condicional) hacer {  
    // bloque instrucciones  
}
```

- Para un **sentencia o estructura de switch**: (Los “parar” son opcionales, a menos que se quiera ejecución exclusiva de casos)

```
segun (expresión) {  
  
    caso valor1:  
        // bloque de instrucciones  
    parar;  
  
    caso valor2:  
        // bloque de instrucciones  
    parar;  
  
    ...  
  
    otro:  
        // bloque de instrucciones  
    parar;  
}
```

BLOQUE

Un bloque es una unidad de código delimitada por llaves. En él se pueden declarar variables locales, procedimientos y escribir las sentencias a ejecutar.

Para declarar variables locales usamos la palabra reservada “**locales**” y, **entre llaves**, declaramos todas las variables como se ha explicado anteriormente. Estas variables solo existen dentro del bloque.

```
{  
    // declaración de variables locales  
    locales {  
        entero x;  
        booleano esPar;  
    }  
    // declaración de procedimientos  
    // sentencias  
}
```

Al igual que con las variables locales, los procedimientos declarados en un bloque solo pueden usarse dentro de él.

PROCEDIMIENTOS

Para definir un procedimiento se hace como en C, pero usando la palabra reservada “**procedimiento**”.

Se pueden pasar parámetros tanto por referencia como por valor, y estos parámetros pueden ser incluso arrays unidimensionales y bidimensionales.

La declaración del procedimiento va seguida de un bloque de instrucciones.

```
procedimiento suma(entero a, entero b, entero & res) {  
    // bloque instrucciones  
}
```

Los procedimientos no devuelven un valor. Su objetivo es ejecutar una secuencia de instrucciones.

Para llamar a un procedimiento ya declarado se hace como en C. Un ejemplo sería:

```
suma(2, 4, sum);
```

PROGRAMA

Los programas empiezan con la palabra reservada “**programa**” y un bloque de instrucciones, dentro del cual se pueden declarar variables y procedimientos y escribir las sentencias a ejecutar.

```
programa {  
    // bloque  
}
```

IDENTIFICACIÓN DE LOS TOKENS

Nombre	Identificador	Atributos	Expresión regular
ID	257	lexema	letra(letra numero)*
TIPO	258		"entero" "real" "caracter" "booleano"
ASIG	259		"=
TRUE	260		"true"
FALSE	261		"false"
PUNTO	262		"."
LLAVED	263		"}"
LLAVEI	264		"{"
OPERUNI	265	0:- 1:++ 2:-- 3:& 4:*	"-" "++" "--" "&" "+"
OPERBIN	266	0:+ 1:- 2:* 3:** 4:/ 5:% 6:> 7:>= 8:< 9:<= 10:+= 11:-= 12:*= 13:/=	"+" "-" "*" "**" "/" "%" ">" ">=" "<" "<=" "+=" "-=" "*=" "/=
OPERLOG	267	0:&& 1: 2:! 3:== 4:!=	"&&" " " "!" "==" "!="
DOSPUNTOS	268		"."
CORD	269		"]"
CORI	270		"["
PUNCOMA	271		"."
LOCALES	272		"locales"
LECTURA	273		"lectura"
ESCRITURA	274		"escritura"
SI	275		"si"
SINO	276		"si no"
MIENTRAS	277		"mientras"

HACER	278		“hacer”
PARAR	279		“parar”
SEGUN	280		“segun”
CASO	281		“caso”
OTRO	282		“otro”
PARAR	283		“parar”
PROG	284		“programa”
PROCD	285		“procedimiento”
CADENA	286		“[[^]]*”
CONSTANTE_REAL	287		([0-9]+\.[0-9]* \.[0-9]+ [0-9]+\.)
CONSTANTE_ENTERA	288		([0-9]+)
COMDOBLEAB	288		““”
COMDOBLECE	289		“””
COMSIMPLE	290		“ ”
COMA	291		“ , ”
PIZQ	292		“(”
PDCHA	293		“) ”
OPERBIT	294		“~”, “ ”, “^”, “<<”, “>>”